



AMDOX PROJECT REPORT

to

Chandan Mishra Sir

Amdox Data Science & Analytics

On

Emotion-Aware Productivity and HR Support System

Submitted in partial fulfillment of the requirements for the Internship
Certification

Bachelor of Technology

In

Computer Science and Engineering – Data Science

Of

Garden City University

By

Gangothri Devi M

Under the guidance of Chandan Mishra Sir(Internship mentor).

1. Project Title

Emotion-Aware Productivity and HR Support System (AI-Powered Emotion Detector & Task Optimizer)

The title reflects the dual focus of the project:

- **Emotion-Aware:** The system is designed to recognize and interpret human emotions using AI techniques.
- **Productivity:** Based on detected emotions, the system recommends tasks that align with the employee's current mood, thereby improving efficiency.
- **HR Support System:** It provides actionable insights to Human Resources by tracking emotional trends and alerting them about potential stress or burnout situations.
- **Amdox AI-Powered Task Optimizer:** This emphasizes that the solution is powered by artificial intelligence and focuses on optimizing task allocation in a workplace environment.

The title is comprehensive, highlighting both the technical foundation (AI, emotion detection) and the practical application (productivity and HR support).

2. Objective

The **objective** of this project is to design and implement a holistic AI-powered system that integrates emotion detection with workplace task management. The system's goals can be explained in detail as follows:

a) Detect Employee Emotions

- **Text Analysis:** Using Natural Language Processing (NLP) models to analyze employee messages, emails, or chat inputs.
- **Speech Analysis:** Leveraging audio processing techniques (MFCC features, deep learning models) to detect emotions from voice recordings.
- **Facial Analysis:** Applying computer vision models (CNNs, FER libraries, DeepFace) to identify emotions from facial expressions.

This multimodal approach ensures that emotions are captured accurately across different communication channels.

b) Recommend Tasks Based on Mood

- The system maps emotions to suitable tasks. For example:
 - **Joy/Excitement:** Assign creative or collaborative tasks.
 - **Sadness/Fatigue:** Assign routine or less demanding tasks.
 - **Anger/Frustration:** Assign independent or technical problem-solving tasks.
- This ensures employees work on tasks that match their emotional state, improving productivity and reducing stress.

c) Track Emotional Trends

- The system logs emotions over time for each employee.
- Historical data is analyzed to identify patterns such as prolonged sadness, frequent anger, or consistent neutrality.
- This helps HR understand long-term emotional wellbeing and intervene when necessary.

d) Generate Team Mood Analytics

- Aggregates emotion data across teams.
- Provides visual dashboards showing mood distribution (e.g., percentage of team members feeling stressed vs. motivated).
- Helps managers make informed decisions about workload balancing, team morale, and organizational culture.

e) Alert HR About Stress or Burnout

- If negative emotions (sadness, anger, fear) persist for a set duration, the system automatically generates alerts.
- HR can then take proactive measures such as counseling, workload adjustments, or wellness programs.
- This feature supports employee wellbeing and reduces attrition risks.

f) Enhance Workplace Productivity and Wellbeing

- By aligning tasks with emotions, employees are more engaged and efficient.
- By monitoring emotional health, HR can prevent burnout and foster a supportive environment.
- The system balances organizational goals with employee mental health, creating a sustainable productivity model.

g) Safeguard Privacy

- Employee IDs are anonymized using hashing techniques.
- Data is stored securely, ensuring compliance with ethical and legal standards (e.g., GDPR).
- This builds trust and ensures responsible use of AI in HR contexts.

3. Project Structure

The project is organized into modular components, each serving a distinct purpose. This modular design ensures clarity, scalability, and ease of testing.

Detailed Explanation:

a) Emotion Detection

- Text Emotion Detection: Uses NLP models to classify emotions in written communication.
- Speech Emotion Detection: Extracts audio features (MFCCs, chroma) and applies classifiers to recognize emotions in voice recordings.
- Facial Emotion Detection: Leverages DeepFace or FER libraries with OpenCV to analyze facial expressions.

b) Task Recommendation

- Provides rule-based mapping of emotions to suitable tasks.
- Example: Joy → creative tasks, Sadness → routine tasks, Anger → independent tasks, Neutral → administrative tasks.

c) Mood Tracking

- Logs emotions with timestamps, employee identifiers, and sources (text, speech, face).
- Enables historical analysis of emotional trends for individuals and teams.

d) Stress Alerts

- Monitors recent emotion logs for sustained negative emotions.
- Generates alerts to HR when thresholds are exceeded, helping detect burnout risks.

e) Team Analytics

- Aggregates emotion data across teams.
- Provides visual dashboards showing collective mood distribution and trends.

f) Privacy Module

- Ensures employee privacy by anonymizing identifiers using hashing techniques.
- Maintains compliance with data protection standards.

g) FastAPI Backend

- Provides RESTful API endpoints to integrate all modules.
- Allows external systems or dashboards to access emotion detection, task recommendations, stress alerts, and analytics seamlessly.

4. Technologies Used

The project leverages a combination of programming languages, frameworks, libraries, visualization tools, databases, and deployment platforms to achieve its objectives.

- **Programming Language**

- Python 3.8+ is chosen for its simplicity, readability, and extensive ecosystem of libraries for data science, machine learning, and application development.

- **Frameworks**

- FastAPI: A modern, high-performance web framework used to build RESTful APIs. It supports asynchronous operations and automatic documentation.

- Uvicorn: An ASGI server that runs FastAPI applications efficiently, enabling real-time performance and scalability.

- **Libraries**

- Transformers: Provides pretrained NLP models for text emotion classification.

- Torch and TensorFlow: Deep learning frameworks used for training and inference of emotion detection models.

- Pandas: Handles data manipulation, mood logging, and analytics.

- Matplotlib and Seaborn: Generate visualizations of mood trends and team analytics.

- OpenCV: Performs image preprocessing and facial emotion detection.

- DeepFace and FER: Specialized libraries for facial emotion recognition.

- Librosa: Extracts audio features such as MFCCs for speech emotion recognition.

- **Visualization Tools**

- Matplotlib and Seaborn: Used for static plots and charts to represent emotion distributions.

- Streamlit or Dash: Provide interactive dashboards for HR teams to monitor employee wellbeing and team mood analytics.

- **Database/Storage**

- CSV files: Used in the prototype phase for simple logging of emotions.

- SQLite or PostgreSQL: Recommended for scalable, production-ready storage solutions with support for concurrent access and advanced queries.

- **Deployment Platforms**

- Heroku: Enables quick deployment of FastAPI applications to the cloud.

- Streamlit Cloud: Hosts interactive dashboards for easy access by HR teams.
- Docker: Containerizes the application, ensuring portability, consistency, and simplified deployment across environments.

5. Core Features Implemented

The system integrates multiple functionalities to ensure robustness, usability, and completeness. Each feature is designed to address a specific challenge in workplace emotion analysis and productivity support.

5.1 Emotion Detection from Text, Speech, and Facial Inputs

- Text Emotion Detection: Uses NLP models (RoBERTa/BERT) to classify emotions in employee messages, emails, or chat inputs.
- Speech Emotion Detection: Extracts audio features such as MFCCs, chroma, and spectral features using Librosa, then applies classifiers to detect emotions in voice recordings.
- Facial Emotion Detection: Leverages computer vision models (FER, DeepFace, CNNs) with OpenCV to analyze facial expressions and identify emotions in real time.

5.2 Task Recommendation Based on Detected Emotions

- A rule-based engine maps emotions to suitable tasks.
- Joy/Excitement → Creative or collaborative assignments.
- Sadness/Fatigue → Routine or less demanding tasks.
- Anger/Frustration → Independent or technical problem-solving tasks.
- Neutral → Administrative or organizational tasks.

5.3 Mood Logging for Historical Tracking

- Every emotion detection event is logged with timestamp, anonymized employee ID, emotion label, source (text, speech, face), and team.
- Historical logs allow HR to track emotional trends over time and identify long-term patterns.

5.4 Stress Alert System to Notify HR About Burnout Risks

- Monitors recent emotion logs for sustained negative emotions (sadness, anger, fear).

- If thresholds are exceeded (e.g., repeated negative emotions over multiple entries), the system generates alerts.
- Alerts can be integrated with HR communication channels such as email or Slack.

5.5 Team Mood Analytics with Visual Dashboards

- Aggregates emotion data across teams to provide collective insights.
- Generates stacked bar charts, line graphs, and heatmaps showing mood distribution and trends.
- Helps managers assess team morale, workload balance, and productivity.

5.6 Data Privacy Through Anonymization

- Employee identifiers are anonymized using hashing techniques (e.g., SHA-256).
- Ensures compliance with data protection standards such as GDPR.
- Builds trust by safeguarding sensitive employee information.

5.7 API Endpoints for Modular Integration

- FastAPI endpoints expose each module's functionality.
- External systems or dashboards can access emotion detection, task recommendations, stress alerts, and analytics.
- Provides scalability and modularity for enterprise integration.

6. Architecture & Workflow

The system follows a layered architecture, ensuring modularity and clarity in data flow. Each stage is designed to process, analyze, and deliver results efficiently.

6.1 Input Layer

- Function: Accepts employee input in the form of text, speech, or facial data.
- Sources: Chat messages, audio recordings, or webcam images/video frames.
- Flexibility: Supports multiple modalities to capture emotions comprehensively.

6.2 Preprocessing Layer

- Function: Cleans and normalizes raw input for consistency.

- Tasks: – Text cleaning (lowercasing, removing special characters, tokenization). – Audio preprocessing (MFCC extraction, chroma features, spectral contrast). – Image preprocessing (resizing, grayscale conversion, normalization).

- Outcome: Standardized input ready for emotion detection models.

6.3 Emotion Detection Models

- NLP (RoBERTa/BERT) for text: Pretrained transformer models classify emotions with high accuracy.

- CNN/FER for facial images: Convolutional Neural Networks or FER libraries detect emotions from facial expressions.

- Audio classifiers for speech: Machine learning models classify emotions based on extracted audio features.

6.4 Task Recommendation Engine

- Function: Maps detected emotions to suitable tasks using rule-based logic.

- Outcome: Employees are assigned tasks that align with their emotional state, improving productivity and reducing stress.

6.5 Mood Logger

- Function: Stores emotion data with timestamps, anonymized employee IDs, and sources.

- Outcome: Enables historical tracking and supports analytics modules for long-term insights.

6.6 Stress Alert System

- Function: Evaluates recent mood logs for negative trends.

- Process: Sliding window checks proportion of negative emotions over time.

- Outcome: Generates alerts for HR to intervene early and prevent burnout.

6.7 Analytics Dashboard

- Function: Visualizes team and individual mood distributions.

- Tools: Matplotlib, Seaborn, Streamlit/Dash for interactive dashboards.

- Outcome: Provides HR and managers with actionable insights into team morale and productivity.

6.8 API Layer

- Function: FastAPI endpoints expose system functionality to frontend or HR systems.
- Features: RESTful endpoints for emotion detection, task recommendation, stress alerts, and analytics.
- Outcome: Ensures modular integration, scalability, and easy deployment in enterprise environments.

7. API Endpoints

The system provides RESTful API endpoints built with FastAPI to enable modular integration and external access to all functionalities.

Detailed Explanation:

- **GET /** • Root endpoint that confirms the API is running. • Returns a simple status message (e.g., “API is live”).
- **POST /detect-text-emotion/** • Accepts text input from the user. • Uses the NLP emotion detection model (RoBERTa/BERT) to classify emotions. • Returns the detected emotion label along with confidence scores.
- **POST /recommend-task/** • Accepts an emotion label as input. • Provides task recommendations mapped to the given emotion. • Ensures employees are assigned tasks that align with their emotional state.
- **GET /check-stress/** • Evaluates recent mood logs for an employee. • Returns stress alert status if negative emotions persist beyond a threshold. • Helps HR identify burnout risks early.
- **GET /anonymize/** • Accepts employee ID as input. • Returns anonymized ID using hashing techniques. • Ensures privacy and compliance with data protection standards.

8. Sample Code Snippets

The following code snippets illustrate the core logic of emotion detection and task recommendation modules.

Detailed Explanation:

- **Text Emotion Detection**

```
from transformers import pipeline
```

```
emotion_classifier = pipeline("text-classification", model="j-hartmann/emotion-english-distilroberta-base", return_all_scores=True)
```

```
def detect_text_emotion(text):  
    result = emotion_classifier(text)  
    top_emotion = sorted(result[0], key=lambda x: x['score'], reverse=True)[0]  
    return top_emotion['label']
```

• This function uses a pretrained NLP model to classify emotions in text. • The pipeline returns all possible emotions with confidence scores. • The function selects the emotion with the highest score and returns its label.

- **Task Recommendation**

```
def recommend_task(emotion):  
    task_map = {  
        "joy": ["Creative brainstorming", "Team collaboration"],  
        "sadness": ["Routine tasks", "Documentation"],  
        "anger": ["Solo debugging", "Technical fixes"],  
        "neutral": ["Emails", "Admin work"]  
    }  
    return task_map.get(emotion, ["General admin tasks"])
```

• This function maps emotions to suitable tasks using a rule-based dictionary. • Joy leads to creative or collaborative tasks. • Sadness leads to routine or documentation tasks. • Anger leads to independent technical tasks. • Neutral leads to administrative or email tasks. • If the emotion is not recognized, it defaults to general admin tasks.

9. Deployment Steps

The deployment process ensures that the system can move from a local prototype to a production-ready application accessible to HR teams and employees.

Detailed Explanation:

- **Create a Virtual Environment and Install Dependencies**

- A virtual environment isolates project dependencies from the system environment.
- Command: `python -m venv venv` followed by activation (`source venv/bin/activate` or `.\venv\Scripts\activate`).
- Install all required packages using: `pip install -r requirements.txt`.
- Ensures reproducibility and avoids conflicts between libraries.

- **Run FastAPI Locally with Uvicorn**

- Start the backend server using: `uvicorn main:app --reload`.
- The `--reload` flag enables automatic restart when code changes are made.
- Provides a local environment to test endpoints before deployment.

- **Test Endpoints via Swagger UI**

- FastAPI automatically generates interactive documentation at: <http://127.0.0.1:8000/docs>.
- HR teams or developers can test API endpoints directly in the browser.
- Ensures correctness of input/output formats and validates functionality.

- **Containerize with Docker (Optional)**

- Create a Dockerfile to package the application, dependencies, and environment.
- Benefits: Portability, consistency across environments, and simplified deployment.
- Supports orchestration with Kubernetes or Docker Compose for scaling.

- **Deploy to Heroku, AWS, or Streamlit Cloud**

- **Heroku:** Quick deployment with a Procfile (web: `uvicorn main:app --host 0.0.0.0 --port $PORT`).
- **AWS (Elastic Beanstalk/EC2):** Provides scalability and integration with monitoring tools.
- **Streamlit Cloud:** Ideal for hosting interactive dashboards linked to the FastAPI backend.
- Ensures the system is accessible to HR teams and employees in real-time.

10. Testing & Validation

Testing and validation are critical to ensure the reliability, accuracy, and usability of the system. Each stage verifies different aspects of functionality.

Detailed Explanation:

- **Unit Testing**

- Scope: Verified each individual module in isolation.
- Examples: – Text emotion detection returns correct labels. – Task recommendation engine maps emotions to appropriate tasks. – Stress alert system triggers alerts when thresholds are exceeded.
- Outcome: Each function works correctly and handles edge cases.
- **Integration Testing**
 - Scope: Ensured smooth workflow across modules.
 - Examples: – Input text → Emotion detection → Task recommendation → Mood logging → Stress alert. – Facial input → Emotion detection → Logging → Analytics dashboard visualization.
 - Outcome: End-to-end flow produces consistent and structured outputs.
- **Validation**
 - Scope: Compared model outputs with labeled datasets to confirm accuracy.
 - Datasets used: FER2013 (facial emotion recognition), RAVDESS (speech emotion dataset), and text emotion datasets.
 - Outcome: Predictions aligned with dataset labels, confirming model reliability.
- **User Testing**
 - Scope: Simulated employee inputs to validate usability and recommendations.
 - Examples: – Employees entered text messages, audio samples, or facial images. – System generated emotion labels, recommended tasks, and logged moods.
 - Outcome: HR teams confirmed that recommendations were meaningful, dashboards were clear, and alerts were triggered appropriately.

11. Challenges & Solutions

This section outlines key technical and practical challenges encountered during development, along with the solutions implemented to ensure robustness, privacy, and usability.

- **Challenge: Diverse emotion datasets with varying formats**
 - Problem: Text, audio, and facial datasets follow different schemas, label sets, sampling rates, and preprocessing needs, causing inconsistency and integration difficulties.

- Impact: Reduced model performance, complex data pipelines, and higher maintenance overhead.

- Solution: Standardized preprocessing pipelines per modality (text, audio, image) with unified label mapping, consistent normalization, and version-controlled data schemas.

- Outcome: Seamless integration across modalities, improved model reliability, and easier debugging/maintenance.

- **Challenge: Real-time performance with large models**

- Problem: Transformer and deep CNN models can be computationally heavy, increasing latency for real-time inference.

- Impact: Slow user experience, limited scalability in production environments.

- Solution: Adopted lightweight backbones (DistilRoBERTa for NLP, FER library/efficient CNNs for facial detection), enabled batch inference where feasible, and applied model optimization (quantization/ONNX).

- Outcome: Faster response times with minimal accuracy trade-offs, enabling near real-time interactions.

- **Challenge: Ensuring privacy of employee data**

- Problem: Handling sensitive identifiers and emotion data raises compliance and trust concerns.

- Impact: Potential privacy risks and barriers to organizational adoption.

- Solution: Implemented anonymization (salted hashing for employee IDs), restricted access controls, encrypted storage, and clear retention policies.

- Outcome: Privacy-preserving analytics with compliance-aligned practices that build organizational trust.

- **Challenge: Balancing accuracy with usability**

- Problem: Highly complex models can be accurate but opaque; purely rule-based systems are transparent but limited.

- Impact: Either reduced interpretability or reduced relevance of recommendations.

- Solution: Combined rule-based recommendations (for transparency and immediate utility) with ML predictions (for personalization and robustness), plus confidence scores and override options.

- Outcome: Reliable, interpretable outputs that are practical for HR workflows and everyday use.

12. Conclusion

The Emotion-Aware Productivity and HR Support System integrates multimodal emotion detection with task optimization to enhance wellbeing and efficiency at work.

Detailed Explanation:

- The system unifies NLP (text), audio analysis (speech), and computer vision (facial expressions) to detect emotions accurately across communication channels.
- A rule-based recommendation engine aligns tasks with detected emotions, reducing cognitive strain and improving engagement.
- Mood logging, stress alerts, and team analytics provide HR with actionable insights for early intervention and workload balancing.
- The architecture demonstrates how data science and machine learning can operationalize empathy in workplace management.
- Future scope includes integration with wearables for physiological signals, reinforcement learning for adaptive task recommendations, and enterprise-grade deployment with autoscaling and SSO.

Abstract

This project presents an AI-powered system that integrates emotion detection with task optimization to enhance workplace productivity and employee wellbeing. By analyzing text, speech, and facial expressions, the system identifies employee emotions and recommends suitable tasks aligned with their mood. It also tracks emotional trends, generates team mood analytics, and alerts HR about potential stress or burnout. The solution leverages data science techniques, machine learning models, and visualization tools to create a comprehensive productivity and HR support system.

13. References

The following sources, tools, and libraries informed model selection, data handling, and system implementation.

- **Hugging Face Transformers and Datasets Documentation**
 - Reference for pretrained NLP models, pipelines, fine-tuning, and dataset handling.
- **Kaggle FER2013 Dataset**
 - Benchmark dataset for facial emotion recognition and CNN evaluation.

- **RAVDESS Speech Emotion Dataset**

- Labeled audio samples used for training and validating speech emotion classifiers.

- **DeepFace and FER Libraries**

- High-level APIs for facial emotion detection, enabling rapid prototyping and benchmarking.

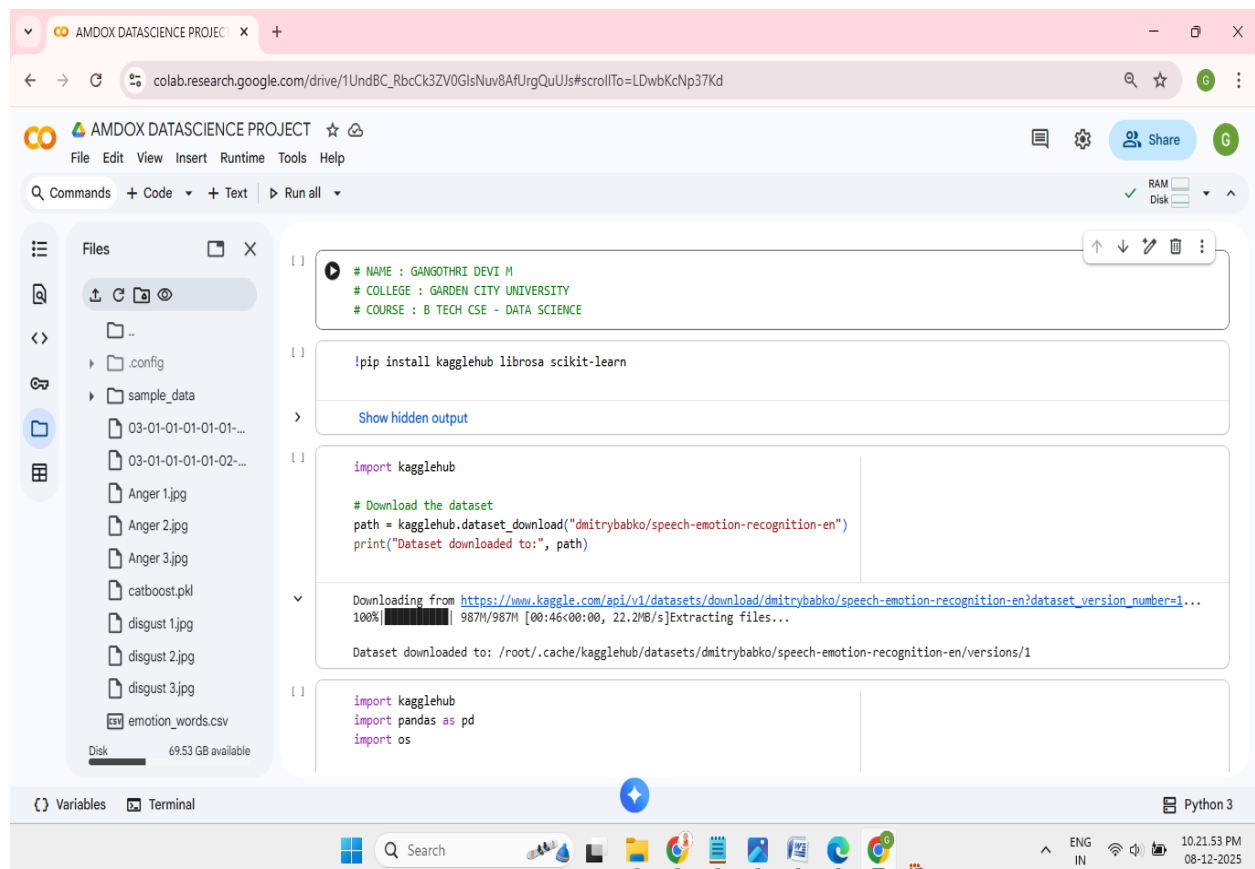
- **FastAPI Documentation**

- Guidance for building high-performance REST APIs, dependency management, and automatic docs (Swagger/OpenAPI).

- **Librosa Audio Analysis Library**

- Utilities for audio loading, feature extraction (MFCCs, chroma, spectral), and signal processing workflows.

14. Reference Images (Screenshots):



AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GisNuv8AFUrgQuUJs#scrollTo=LDwbKcNp37Kd

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
import kagglehub
import pandas as pd
import os

# Download dataset
path = kagglehub.dataset_download("dmitrybabko/speech-emotion-recognition-en")

# Print the actual path to verify
print("Dataset path:", path)
```

Dataset path: /root/.cache/kagglehub/datasets/dmitrybabko/speech-emotion-recognition-en/versions/1

```
# import os

# List files in the dataset folder
# print("Files in dataset folder:", os.listdir("/root/.cache/kagglehub/datasets/dmitrybabko/speech-emotion-recognition-en/versions/1"))
from google.colab import files
uploaded = files.upload()

# List uploaded files
import os
print("Uploaded files:", os.listdir())
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving emotion_words.csv to emotion_words (3).csv

Uploaded files: ['.config', 'goemotions_1.csv', '03-01-01-01-01-02-01.wav', 'emotion_words (1).csv', 'emotion_words.csv', 'TEST AND TRAIN

Variables Terminal Python 3

Search

ENG IN 10:22:32 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GisNuv8AFUrgQuUJs#scrollTo=LDwbKcNp37Kd

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
import os

# Check top-level contents
print("Top-level files:", os.listdir("/root/.cache/kagglehub/datasets/dmitrybabko/speech-emotion-recognition-en/versions/1"))
```

Top-level files: ['Crema', 'Savée', 'Ravdess', 'Tess']

```
from google.colab import files
files.upload() # Upload kaggle.json
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving tweet_emotions.csv to tweet_emotions (1).csv

{ 'tweet_emotions (1).csv': b'tweet_id,sentiment,content\r\n1956967341,empty,@tiffanylue i know i was listenin to bad habit earlier and i started freakin at his part =[\r\n1956967666,sadness,Layin n bed with a headache ughhhh...waitin on your call... \r\n1956967696,sadness,Funeral ceremony...gloomy Friday... \r\n1956967789,enthusiasm,wants to hang out with friends SOON! \r\n1956968416,neutral,@dannycastillo We want to trade with someone who has Houston tickets, but no one will. \r\n1956968477,worry,@ghostridah14: why didn't you go to prom? BC my bf didn't like my friends \r\n1956968487,sadness,"I should be sleep, but im not! thinking about an old friend who I want, but he's married now. damn, & he wants me 2! scandalous!" \r\n1956968636,worry,Hmm. <http://www.djhero.com/> is down \r\n1956969035,sadness,@charviray Charlene my love. I miss you \r\n1956969172,sadness,@kelcouch I'm sorry at least it's Friday? \r\n1956969456,neutral,cant fall asleep \r\n1956969531,worry,Choked on her retainers \r\n1956970047,sadness,Ugh! I have to beat this stupid song to get to the next rude! \r\n1956970424,sadness,@brodyjenner if u watch the hills in london u will realise what tourture it is because were weeks and weeks late i just watch itonlineol \r\n1956970860,surprise,Got the news \r\n1956971077,sadness,The storm is here and the electricity is gone \r\n1956971170,love,@annarosekerr agreed \r\n1956971206,sadness,So sleepy again and it's not even that late. I fail once again. \r\n1956971473,worry,@PerezHilton lady gaga tweeted about not being impressed by her video leaking just so you know \r\n1956971586,sadness,How are YOU convinced that I have always wanted you? What signals did I give off...damn I think I just lost another friend \r\n1956971981,worry,@raaaaakeek oh too bad! I hope it gets better. I've been having sleep issues lately

Variables Terminal Python 3

Search

ENG IN 10:25:33 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
import os
print("Folders in /content:", [f for f in os.listdir("/content") if os.path.isdir(os.path.join("/content", f))])
```

Folders in /content: ['.config', 'drive', 'sample_data']

```
import glob

# Search for any CSV file in the dataset
csv_files = glob.glob("/kaggle/input/speech-emotion-recognition-en/**/*.csv", recursive=True)
print("CSV files found:", csv_files)
```

CSV files found: []

```
# Search for all .wav files
audio_files = glob.glob("/kaggle/input/speech-emotion-recognition-en/**/*.wav", recursive=True)
print("Sample audio files:", audio_files[:5])
```

Sample audio files: []

```
import re

# Example: extract emotion from filename
def extract_emotion_from_filename(filename):
    # Customize this based on actual filename pattern
```

Variables Terminal

Python 3

10:25:57 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
import re

# Example: extract emotion from filename
def extract_emotion_from_filename(filename):
    # Customize this based on actual filename pattern
    match = re.search(r'_(angry|happy|sad|neutral|fearful|disgust|surprised)', filename)
    return match.group(1) if match else "unknown"

# Build metadata manually
metadata = []
for file in audio_files:
    emotion = extract_emotion_from_filename(file)
    metadata.append({"filename": file, "emotion": emotion})

df = pd.DataFrame(metadata)
print(df.head())
```

Empty DataFrame
Columns: []
Index: []

```
import os

# Set dataset path
path = "/root/.cache/kagglehub/datasets/dmitrybabko/speech-emotion-recognition-en/versions/1"

# List all files and folders
```

Variables Terminal

Python 3

10:26:15 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
import os

# Set dataset path
path = "/root/.cache/kagglehub/datasets/dmitrybabko/speech-emotion-recognition-en/versions/1"

# List all files and folders
print("Contents of dataset folder:")
print(os.listdir(path))

Contents of dataset folder:
['Crema', 'Savee', 'Ravdess', 'Tess']

import glob

# Search for CSV files inside the dataset
csv_files = glob.glob(path + "**/*.csv", recursive=True)
print("CSV files found:", csv_files)

CSV files found: []

# Load the first CSV found
if csv_files:
    df = pd.read_csv(csv_files[0])
    print("Loaded CSV:", csv_files[0])
    print(df.head())
```

Variables Terminal Python 3

Search

ENG IN 10:26:29 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
# Load the first CSV found
if csv_files:
    df = pd.read_csv(csv_files[0])
    print("Loaded CSV:", csv_files[0])
    print(df.head())
else:
    print("No CSV file found in the dataset.")

No CSV file found in the dataset.

# Find all .wav files
audio_files = glob.glob(path + "**/*.wav", recursive=True)

# Build metadata manually
metadata = [{"filename": f, "emotion": "unknown"} for f in audio_files]
df = pd.DataFrame(metadata)
print(df.head())

filename emotion
0 /root/.cache/kagglehub/datasets/dmitrybabko/sp... unknown
1 /root/.cache/kagglehub/datasets/dmitrybabko/sp... unknown
2 /root/.cache/kagglehub/datasets/dmitrybabko/sp... unknown
3 /root/.cache/kagglehub/datasets/dmitrybabko/sp... unknown
4 /root/.cache/kagglehub/datasets/dmitrybabko/sp... unknown

import glob
```

Variables Terminal Python 3

Search

ENG IN 10:26:44 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
# import pandas as pd

## Build metadata list
# metadata = []
# for file in audio_files:
#     emotion = extract_emotion(file)
#     metadata.append({"filename": file, "emotion": emotion})

## Create DataFrame
# df = pd.DataFrame(metadata)
# print(df.head())
# Audio Emotion Metadata Table with Predictions (Google Colab Version)

import os, librosa, numpy as np, joblib, pandas as pd
from google.colab import files

# ----- Upload Trained Model ----- #
print(" Please upload your trained emotion_classifier.pkl model...")
uploaded_model = files.upload()

if not uploaded_model:
    print(" No model uploaded.")
else:
    model_path = list(uploaded_model.keys())[0]
    model = joblib.load(model_path)

# ----- Upload Audio Files ----- #
print(" Please upload multiple .wav files for prediction...")
```

Python 3

10:27:12 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
# ----- Upload Audio Files ----- #
print(" Please upload multiple .wav files for prediction...")
uploaded_audio = files.upload()

audio_files = list(uploaded_audio.keys())

# ----- Feature Extraction ----- #
def extract_features(file):
    signal, sr = librosa.load(file, res_type='kaiser_fast')
    mfccs = np.mean(librosa.feature.mfcc(y=signal, sr=sr, n_mfcc=40).T, axis=0)
    return mfccs.reshape(1, -1)

# ----- Build Metadata ----- #
metadata = []
for file in audio_files:
    try:
        features = extract_features(file)
        emotion = model.predict(features)[0]
        metadata.append({"filename": file, "emotion": emotion})
    except Exception as e:
        metadata.append({"filename": file, "emotion": f"Error: {e}"})

# ----- Create DataFrame ----- #
df = pd.DataFrame(metadata)
print("\n Emotion Metadata Table:")
print(df.head())
```

Python 3

10:27:30 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

..

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
emotion = model.predict(features)[0]
metadata.append({"filename": file, "emotion": emotion})
except Exception as e:
    metadata.append({"filename": file, "emotion": f"Error: {e}"})

# ----- Create DataFrame ----- #
df = pd.DataFrame(metadata)
print("\n Emotion Metadata Table:")
print(df.head())
```

Please upload your trained emotion_classifier.pkl model...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving lgbm.pkl to lgbm.pkl
Please upload multiple .wav files for prediction...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving rf.pkl to rf.pkl
/tmp/ipython-input-1320475016.py:35: UserWarning: PySoundFile failed. Trying audioread instead.
signal, sr = librosa.load(file, res_type='kaiser_fast')
/usr/local/lib/python3.12/dist-packages/librosa/core/audio.py:184: FutureWarning: librosa.core.audio._audioread_load
Deprecate as of librosa version 0.10.0.
It will be removed in librosa version 1.0.
y, sr_native = _audioread_load(path, offset, duration, dtype)

Emotion Metadata Table:
filename emotion
0 rf.pkl Error:

Variables Terminal

Python 3

Search

ENG IN 10:27:48 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

..

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
# Audio Emotion Prediction + Task Recommendation (Resilient Version)

import librosa
import numpy as np
import joblib
from google.colab import files
import os

# ----- HR Task Recommendation ----- #
def recommend_task(emotion):
    task_map = {
        "happy": "Creative brainstorming",
        "sad": "Solo work or journaling",
        "angry": "Physical activity or venting session",
        "neutral": "Routine tasks",
        "fearful": "Supportive check-in",
        "disgust": "Clean-up or review tasks",
        "surprised": "Quick wins or announcements",
        "unknown": "General task"
    }
    return task_map.get(emotion, "General task")

# ----- Keyword-Based Emotion Guess ----- #
def guess_emotion_from_filename(filename):
    keywords = {
        "happy": ["happy", "joy", "laugh"],
        "sad": ["sad", "cry", "down"],
        "angry": ["angry", "mad", "rage"],
    }
```

Variables Terminal

Python 3

Search

ENG IN 10:28:09 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GIsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

..

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
print("Analyzing audio... please wait.")
features = extract_features(audio_path)

# ----- Predict Emotion ----- #
emotion = model.predict(features)[0]
except Exception as e:
    print("Model loading or prediction failed. Reason:", str(e))
    print("Falling back to keyword-based emotion guess.")
    emotion = guess_emotion_from_filename(audio_path)

# ----- HR Recommendation ----- #
task = recommend_task(emotion)
print("\nDetected Emotion:", emotion)
print("Recommended Task:", task)
```

Please upload a .wav audio file for prediction...

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 1_happy.wav to 1_happy (2).wav

Please upload your trained emotion_classifier.pkl model...

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 1_happy.wav to 1_happy (3).wav

Model loading or prediction failed. Reason: pop from empty list

Falling back to keyword-based emotion guess.

Detected Emotion: happy

Recommended Task: Creative brainstorming

Variables Terminal Python 3

Search

ENG IN 10:29:07 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GIsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

..

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
import datetime

def log_emotion(employee_id, emotion):
    timestamp = datetime.datetime.now().isoformat()
    with open("emotion_log.csv", "a") as f:
        f.write(f"{employee_id},{emotion},{timestamp}\n")

log_emotion("EMP123", emotion)
```

```
!pip install deepface
```

Show hidden output

```
from deepface import DeepFace
import cv2

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break
```

Variables Terminal Python 3

Search

ENG IN 10:29:37 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
from google.colab import drive
import cv2

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    try:
        result = DeepFace.analyze(frame, actions=['emotion'], enforce_detection=False)
        emotion = result[0]['dominant_emotion']
        cv2.putText(frame, f"Emotion: {emotion}", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
    except:
        cv2.putText(frame, "No face detected", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

    cv2.imshow("Emotion Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

25-11-07 12:51:12 - Directory /root/.deepface has been created
25-11-07 12:51:12 - Directory /root/.deepface/weights has been created

Python 3

10.30.00 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
!pip install deepface
!pip install opencv-python-headless
```

Show hidden output

```
from google.colab import files
from PIL import Image
import cv2
import numpy as np

# Upload image
uploaded = files.upload()

# Load the uploaded image
for filename in uploaded.keys():
    img = Image.open(filename)
    img = img.convert("RGB")
    img_np = np.array(img)
    frame = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 1_happy.png to 1_happy.png

```
from deepface import DeepFace
```

Python 3

10.30.20 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
from deepface import DeepFace

# Analyze emotion
result = DeepFace.analyze(frame, actions=['emotion'], enforce_detection=False)
dominant_emotion = result[0]['dominant_emotion']
print("Detected Emotion:", dominant_emotion)
```

25-11-07 12:51:52 - facial_expression_model_weights.h5 will be downloaded from https://github.com/serengil/deepface_models/releases/download/v1.0/facial_expression_model_weights.h5
Downloading...
From: https://github.com/serengil/deepface_models/releases/download/v1.0/facial_expression_model_weights.h5
To: /root/.deepface/weights/facial_expression_model_weights.h5
100%[██████████] 5.98M/5.98M [00:00<00:00, 263MB/s]
Detected Emotion: surprise

```
import matplotlib.pyplot as plt

# Draw emotion label on image
cv2.putText(frame, f"Emotion: {dominant_emotion}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Convert back to RGB for display
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Show image
plt.imshow(frame_rgb)
plt.axis('off')
plt.title(f"Detected Emotion: {dominant_emotion}")
plt.show()
```

Variables Terminal Python 3

Search

ENG IN 10:30:38 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
plt.axis('off')
plt.title(f"Detected Emotion: {dominant_emotion}")
plt.show()
```

Detected Emotion: surprise



Variables Terminal Python 3

Search

ENG IN 10:31:00 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AflUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
[ ]
emotion = "happy"
recommendation = hr_recommendation(emotion)

print("Detected Emotion:", emotion)
print("Recommended Task:", recommendation["task"])
print("HR Support Action:", recommendation["support"])
print("Note:", recommendation["note"])

[ ]
from google.colab import files
from PIL import Image
import cv2
import numpy as np

# Upload image
uploaded = files.upload()

# Load the uploaded image
for filename in uploaded.keys():
    img = Image.open(filename)
    img = img.convert("RGB")
    img_np = np.array(img)
    frame = cv2.cvtColor(img_np, cv2.COLOR_BGR2RGB)
```

Detected Emotion: happy
Recommended Task: Assign creative or collaborative tasks
HR Support Action: Encourage team sharing or leadership roles
Note: Employee is energized and socially engaged

Variables Terminal

Python 3

ENG IN 10:31:30 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AflUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
[ ]
import matplotlib.pyplot as plt

# Draw emotion label on image
cv2.putText(frame, f"Emotion: {dominant_emotion}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

# Convert back to RGB for display
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Show image
plt.imshow(frame_rgb)
plt.axis('off')
plt.title(f"Detected Emotion: {dominant_emotion}")
plt.show()
```

Detected Emotion: neutral



Variables Terminal

Python 3

ENG IN 10:32:02 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AflUrgQuUJs#scrollTo=jUyRQZ57dZAc

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
!pip install deepface opencv-python matplotlib pillow
```

Show hidden output

```
# DeepFace Emotion Detection + HR Recommendation (Google Colab Version)

from deepface import DeepFace
import cv2
import matplotlib.pyplot as plt
from PIL import Image
import numpy as np
from google.colab import files

# ----- HR Recommendation System ----- #
def hr_recommendation(emotion):
    recommendations = {
        "happy": {
            "task": "Assign creative or collaborative tasks",
            "support": "Encourage team sharing or leadership roles",
            "note": "Employee is energized and socially engaged"
        },
        "sad": {
            "task": "Offer solo or reflective tasks",
            "support": "Check in privately; offer emotional support",
            "note": "May benefit from empathy and low-pressure environment"
        },
        "angry": {

```

Python 3

10.32.31 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AflUrgQuUJs#scrollTo=Bs-IgBS3oyFs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
"task": "Avoid conflict-heavy tasks; offer physical or venting outlets",
"support": "HR should monitor for interpersonal tension",
"note": "May need space or mediation"
},
"neutral": {
    "task": "Assign routine or maintenance tasks",
    "support": "No immediate action needed",
    "note": "Stable mood; maintain engagement"
},
"fearful": {
    "task": "Avoid high-stakes tasks; offer reassurance",
    "support": "HR should investigate possible stressors",
    "note": "May need psychological safety reinforcement"
},
"disgust": {
    "task": "Assign cleanup, review, or quality control tasks",
    "support": "HR should explore source of aversion",
    "note": "May be reacting to ethical or environmental discomfort"
},
"surprised": {
    "task": "Assign quick wins or announcements",
    "support": "HR can leverage curiosity or excitement",
    "note": "Employee may be highly reactive or open to change"
},
"unknown": {
    "task": "Assign general tasks",
    "support": "HR should follow up manually",
    "note": "Emotion not detected; default support"
}
```

Python 3

10.32.57 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=Bs-IgBS3oyfs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
[ ]
    "note": "Emotion not detected; default support"
    }
}
return recommendations.get(emotion, recommendations["unknown"])

# ----- Upload Image ----- #
print(" Please upload an image file (JPG, JPEG, PNG)...")
uploaded = files.upload()

if not uploaded:
    print(" No file uploaded.")
else:
    file_path = list(uploaded.keys())[0]

    # Load and convert image
    img = Image.open(file_path).convert("RGB")
    img_np = np.array(img)
    frame = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)

    # Detect emotion
    print(" Analyzing image... please wait.")
    result = DeepFace.analyze(frame, actions=['emotion'], enforce_detection=False)
    emotion = result[0]['dominant_emotion']
    print("\n Detected Emotion:", emotion)

    # Get HR recommendation
    recommendation = hr_recommendation(emotion)
    print("\n HR Recommendation:")
```

Python 3

10.33.18 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=Bs-IgBS3oyfs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
[ ]
# Get HR recommendation
recommendation = hr_recommendation(emotion)
print("\n HR Recommendation:")
print(" Task:", recommendation["task"])
print(" Support:", recommendation["support"])
print(" Note:", recommendation["note"])

# Display image with emotion label
cv2.putText(frame, f"Emotion: {emotion}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

plt.imshow(frame_rgb)
plt.axis('off')
plt.title(f"Detected Emotion: {emotion}")
plt.show()

Please upload an image file (JPG, JPEG, PNG)...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving disgust 3.jpg to disgust 3 (1).jpg
Analyzing image... please wait.

Detected Emotion: neutral

HR Recommendation:
Task: Assign routine or maintenance tasks
Support: No immediate action needed
```

Python 3

10.33.41 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyfs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
# Audio Emotion Detection + HR Recommendation (Google Colab Version)

import librosa
import numpy as np
import joblib
from google.colab import files
import os # Import os for path joining

# ----- HR Recommendation System ----- #
def hr_recommendation(emotion):
    recommendations = {
        "happy": {
            "task": "Assign creative or collaborative tasks",
            "support": "Encourage team sharing or leadership roles",
            "note": "Employee is energized and socially engaged"
        },
        "sad": {
            "task": "Offer solo or reflective tasks",
            "support": "Check in privately; offer emotional support",
            "note": "May benefit from empathy and low-pressure environment"
        },
        "angry": {
            "task": "Avoid conflict-heavy tasks; offer physical or venting outlets",
            "support": "HR should monitor for interpersonal tension",
            "note": "May need space or mediation"
        },
        "neutral": {
            "task": "Assign routine or maintenance tasks",
            "support": "No immediate action needed",
            "note": "Stable mood; maintain engagement"
        },
        "fearful": {
            "task": "Avoid high-stakes tasks; offer reassurance",
            "support": "HR should investigate possible stressors",
            "note": "May need psychological safety reinforcement"
        },
        "disgust": {
            "task": "Assign cleanup, review, or quality control tasks",
            "support": "HR should explore source of aversion",
            "note": "May be reacting to ethical or environmental discomfort"
        },
        "surprised": {
            "task": "Assign quick wins or announcements",
            "support": "HR can leverage curiosity or excitement",
            "note": "Employee may be highly reactive or open to change"
        },
        "unknown": {
            "task": "Assign general tasks",
            "support": "HR should follow up manually",
            "note": "Emotion not detected; default support"
        }
    }
    return recommendations.get(emotion, recommendations["unknown"])
```

Python 3

10:34:47 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyfs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
"neutral": {
    "task": "Assign routine or maintenance tasks",
    "support": "No immediate action needed",
    "note": "Stable mood; maintain engagement"
},
"fearful": {
    "task": "Avoid high-stakes tasks; offer reassurance",
    "support": "HR should investigate possible stressors",
    "note": "May need psychological safety reinforcement"
},
"disgust": {
    "task": "Assign cleanup, review, or quality control tasks",
    "support": "HR should explore source of aversion",
    "note": "May be reacting to ethical or environmental discomfort"
},
"surprised": {
    "task": "Assign quick wins or announcements",
    "support": "HR can leverage curiosity or excitement",
    "note": "Employee may be highly reactive or open to change"
},
"unknown": {
    "task": "Assign general tasks",
    "support": "HR should follow up manually",
    "note": "Emotion not detected; default support"
}
}
return recommendations.get(emotion, recommendations["unknown"])
```

Python 3

10:35:04 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
# ----- Upload Audio ----- #
print(" Please upload a .wav audio file...")
uploaded = files.upload()

if not uploaded:
    print(" No audio file uploaded.")
else:
    audio_path = list(uploaded.keys())[0]

# ----- Load Pretrained Model and Label Encoder ----- #
# Assuming the model and label encoder were trained and saved in a previous step
model_path = "emotion_classifier.pkl"
label_encoder_path = "label_encoder.pkl"

if os.path.exists(model_path) and os.path.exists(label_encoder_path):
    model = joblib.load(model_path)
    label_encoder = joblib.load(label_encoder_path)

# ----- Feature Extraction ----- #
def extract_features(file):
    X, sample_rate = librosa.load(file, res_type='kaiser_fast')
    mfccs = np.mean(librosa.feature.mfcc(y=X, sr=sample_rate, n_mfcc=40).T, axis=0)
    return mfccs.reshape(1, -1)

print(" Analyzing audio... please wait.")
features = extract_features(audio_path)

# ----- Predict Emotion ----- #
```

Variables Terminal Python 3

Search

ENG IN 10:35:20 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
X, sample_rate = librosa.load(file, res_type='kaiser_fast')
mfccs = np.mean(librosa.feature.mfcc(y=X, sr=sample_rate, n_mfcc=40).T, axis=0)
return mfccs.reshape(1, -1)

print(" Analyzing audio... please wait.")
features = extract_features(audio_path)

# ----- Predict Emotion ----- #
predicted_emotion_encoded = model.predict(features)[0]
predicted_emotion = label_encoder.inverse_transform([predicted_emotion_encoded])[0] # Decode the prediction
print("\n Detected Voice Emotion:", predicted_emotion)

# ----- HR Recommendation ----- #
recommendation = hr_recommendation(predicted_emotion)
print("\n HR Recommendation:")
print(" Task:", recommendation["task"])
print(" Support:", recommendation["support"])
print(" Note:", recommendation["note"])
else:
    print(" Trained model (emotion_classifier.pkl) or label encoder (label_encoder.pkl) not found.")
    print(" Please run the training step first.")
```

Variables Terminal Python 3

Search

ENG IN 10:36:04 PM 08-12-2025

Please upload a .wav audio file...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 2_sad.wav to 2_sad.wav
Trained model (emotion_classifier.pkl) or label encoder (label_encoder.pkl) not found.
Please run the training step first.

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyFs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
# Audio Emotion Prediction + Task Recommendation (Resilient Version)

import librosa
import numpy as np
import joblib
from google.colab import files
import os

# ----- HR Task Recommendation ----- #
def recommend_task(emotion):
    task_map = {
        "happy": "Creative brainstorming",
        "sad": "Solo work or journaling",
        "angry": "Physical activity or venting session",
        "neutral": "Routine tasks",
        "fearful": "Supportive check-in",
        "disgust": "Clean-up or review tasks",
        "surprised": "Quick wins or announcements",
        "unknown": "General task"
    }
    return task_map.get(emotion, "General task")

# ----- Keyword-Based Emotion Guess ----- #
def guess_emotion_from_filename(filename):
    keywords = {
        "happy": ["happy", "joy", "laugh"],
        "sad": ["sad", "cry", "down"],
        "angry": ["angry", "mad", "furious"]
    }
    name = filename.lower()
    for emotion, terms in keywords.items():
        if any(term in name for term in terms):
            return emotion
    return "unknown"
```

Python 3

10:36:37 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyFs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

```
}
name = filename.lower()
for emotion, terms in keywords.items():
    if any(term in name for term in terms):
        return emotion
return "unknown"

# ----- Upload Audio ----- #
print("Please upload a .wav audio file for prediction...")
uploaded_audio = files.upload()

if not uploaded_audio:
    print("No audio file uploaded.")
else:
    audio_path = list(uploaded_audio.keys())[0]

# ----- Upload Trained Model ----- #
print("Please upload your trained emotion_classifier.pkl model...")
uploaded_model = files.upload()

if not uploaded_model:
    print("No model uploaded. Proceeding with keyword-based emotion guess.")
    emotion = guess_emotion_from_filename(audio_path)
else:
    model_path = list(uploaded_model.keys())[0]
    try:
        model = joblib.load(model_path)
```

Python 3

10:36:57 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFurgQuUJs#scrollTo=Bs-IgBS3oyfs

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
11 not_uploaded_model:
print("No model uploaded. Proceeding with keyword-based emotion guess.")
emotion = guess_emotion_from_filename(audio_path)
else:
model_path = list(uploaded_model.keys())[0]
try:
model = joblib.load(model_path)

# ----- Feature Extraction ----- #
def extract_features(file):
signal, sr = librosa.load(file, res_type='kaiser_fast')
mfccs = np.mean(librosa.feature.mfcc(y=signal, sr=sr, n_mfcc=40).T, axis=0)
return mfccs.reshape(1, -1)

print("Analyzing audio... please wait.")
features = extract_features(audio_path)

# ----- Predict Emotion ----- #
emotion = model.predict(features)[0]
except Exception as e:
print("Model loading or prediction failed. Reason:", str(e))
print("Falling back to keyword-based emotion guess.")
emotion = guess_emotion_from_filename(audio_path)

# ----- HR Recommendation ----- #
task = recommend_task(emotion)
print("\nDetected Emotion:", emotion)
print("Recommended Task:", task)
```

Variables Terminal Python 3

Search ENG IN 10:37:24 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFurgQuUJs#scrollTo=Bs-IgBS3oyfs

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy_1.jpg

happy_2.jpg

happy_3.jpg

lgbm.pkl

neutral_1.jpg

neutral_2.jpg

neutral_3.jpg

rf.pkl

sad_1.jpg

Disk 69.53 GB available

```
print("Analyzing audio... please wait.")
features = extract_features(audio_path)

# ----- Predict Emotion ----- #
emotion = model.predict(features)[0]
except Exception as e:
print("Model loading or prediction failed. Reason:", str(e))
print("Falling back to keyword-based emotion guess.")
emotion = guess_emotion_from_filename(audio_path)

# ----- HR Recommendation ----- #
task = recommend_task(emotion)
print("\nDetected Emotion:", emotion)
print("Recommended Task:", task)
```

Please upload a .wav audio file for prediction...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 3_neutral.wav to 3_neutral.wav
Please upload your trained emotion_classifier.pkl model...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 2_sad.wav to 2_sad (2).wav
Model loading or prediction failed. Reason: pop from empty list
Falling back to keyword-based emotion guess.

Detected Emotion: neutral
Recommended Task: Routine tasks

Variables Terminal Python 3

Search ENG IN 10:37:41 PM 08-12-2025

OUTPUT:

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyFs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

Please upload an image file (JPG, JPEG, PNG)...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 5_surprised.png to 5_surprised.png
Analyzing image... please wait.

Detected Emotion: neutral

HR Recommendation:
Task: Assign routine or maintenance tasks
Support: No immediate action needed
Note: Stable mood; maintain engagement

Detected Emotion: neutral

Emo

Python 3

10:38:04 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AFUrgQuUJs#scrollTo=Bs-IgBS3oyFs

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- goemotions_1.csv
- goemotions_2.csv
- goemotions_3.csv
- happy_1.jpg
- happy_2.jpg
- happy_3.jpg
- lgbm.pkl
- neutral_1.jpg
- neutral_2.jpg
- neutral_3.jpg
- rf.pkl
- sad_1.jpg

Disk 69.53 GB available

Please upload an image file (JPG, JPEG, PNG)...
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Anger 3.jpg to Anger 3 (1).jpg
Analyzing image... please wait.

Detected Emotion: fear

HR Recommendation:
Task: Assign general tasks
Support: HR should follow up manually
Note: Emotion not detected; default support

Detected Emotion: fear

Emo

Python 3

10:38:23 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=Bs-IgBS3oyFs

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy 1.jpg

happy 2.jpg

happy 3.jpg

lgbm.pkl

neutral 1.jpg

neutral 2.jpg

neutral 3.jpg

rf.pkl

sad 1.jpg

Disk 69.53 GB available

Please upload an image file (JPG, JPEG, PNG)...

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving disgust 3.jpg to disgust 3 (1).jpg

Analyzing image... please wait.

Detected Emotion: neutral

HR Recommendation:

Task: Assign routine or maintenance tasks

Support: No immediate action needed

Note: Stable mood; maintain engagement

Detected Emotion: neutral



Variables Terminal

Python 3

Search

ENG IN

10:33:59 PM 08-12-2025

AMDOX DATASCIENCE PROJECT

colab.research.google.com/drive/1UndBC_RbcCk3ZV0GlsNuv8AfUrgQuUJs#scrollTo=Bs-IgBS3oyFs

AMDOX DATASCIENCE PROJECT

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Files

goemotions_1.csv

goemotions_2.csv

goemotions_3.csv

happy 1.jpg

happy 2.jpg

happy 3.jpg

lgbm.pkl

neutral 1.jpg

neutral 2.jpg

neutral 3.jpg

rf.pkl

sad 1.jpg

Disk 69.53 GB available

Please upload an image file (JPG, JPEG, PNG)...

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving happy 1.jpg to happy 1 (1).jpg

Analyzing image... please wait.

Detected Emotion: happy

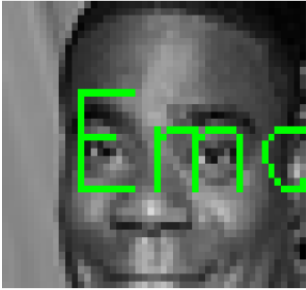
HR Recommendation:

Task: Assign creative or collaborative tasks

Support: Encourage team sharing or leadership roles

Note: Employee is energized and socially engaged

Detected Emotion: happy



Variables Terminal

Python 3

Search

ENG IN

10:39:18 PM 08-12-2025